

Laboratório SVM

1 Máquina de Vetor de Suporte - Support Vector Machines (SVM)

Este notebook materializa os conceitos teóricos do algoritmo SVM, conectando as equações matemáticas aos resultados visuais. Cada bloco de código está comentado linha a linha.

O que faremos hoje?

1. **A Ruína da Escala:** Ver e calcular o colapso da margem geométrica $M = 2/||w||$ sem o StandardScaler.
2. **Margem Suave e Esparsidade:** Medir a contração matemática da margem quando alteramos o hiperparâmetro C.
3. **Teorema de Cover e o Kernel Trick:** Otimizar C e gamma visualizando o relevo das funções de decisão.
4. **Tubo de Insensibilidade:** Provar a isenção de multas dentro do tubo contínuo da regressão epsilon-SVR.

A função `plot_svm_decision_boundary` extrai o hiperplano matemático calculado pelo modelo e desenha.

```
[57]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import make_blobs, make_circles
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC, SVR
from sklearn.metrics import accuracy_score, classification_report, \
    mean_squared_error

sns.set_theme(style='whitegrid')
plt.rcParams['figure.dpi'] = 100

def plot_svm_decision_boundary(clf, X, y, ax, title='Fronteira de Decisão SVM'):

    x_min, x_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
```

```

y_min, y_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 300), np.linspace(y_min,
↪y_max, 300))

Z = clf.decision_function(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

ax.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.coolwarm, s=40,
↪edgecolors='k', linewidth=0.5, alpha=0.85)

ax.contour(xx, yy, Z, colors=['#e65100', '#0f2043', '#009688'],
           levels=[-1.0, 0.0, 1.0], linestyles=['--', '-', '--'],
↪linewidths=[1.5, 2.2, 1.5])

sv = clf.support_vectors_
ax.scatter(sv[:, 0], sv[:, 1], s=120, facecolors='none', edgecolors='red',
↪linewidths=1.8, label=f'SVs Ativos ({len(sv)})')

ax.set_title(title, fontsize=11, fontweight='bold')
ax.legend(loc='upper right', fontsize=8)

```

1.1 Calculando a Margem $M = 2/\|w\|$

Se não padronizarmos os dados, a matemática tentará calcular distâncias usando eixos com tamanhos nominais diferentes. O eixo maior engole o cálculo da norma do vetor w .

```

[58]: np.random.seed(42)
idade = np.random.uniform(18, 65, 200)
renda = np.random.uniform(10000, 150000, 200)
y_escal = np.where((renda / 3000) + idade > 35, 1, -1)
X_escal = np.column_stack((idade, renda))

svm_cru = SVC(kernel='linear', C=1.0)
svm_cru.fit(X_escal, y_escal)

scaler = StandardScaler()
X_escalado = scaler.fit_transform(X_escal)
svm_padronizado = SVC(kernel='linear', C=1.0)
svm_padronizado.fit(X_escalado, y_escal)

w_padronizado = svm_padronizado.coef_[0]
norma_w = np.linalg.norm(w_padronizado)
margem_M = 2 / norma_w

print(f'Linha 1 Crua      : Idade {X_escal[0,0]:.1f} | Renda {X_escal[0,1]:.
↪1f}')

```

```

print(f'Linha 1 Escalonada : Idade {X_escalonado[0,0]:.3f} | Renda_
↳{X_escalonado[0,1]:.3f}\n')

print(f'Sobrecarga de RAM SEM Padronizacao : {len(svm_cru.support_)} Vetores de_
↳Suporte (Travamento)')
print(f'Eficiencia COM Padronizacao          : {len(svm_padronizado.support_)}_
↳Vetores de Suporte (Limpo)')
print(f'Largura Geometrica (M) Padronizada : {margem_M:.4f} unidades\n')

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

ax1.scatter(X_escalado[y_escalado==1, 0], X_escalado[y_escalado==1, 1],_
↳color='#009688', label='Classe +1')
ax1.scatter(X_escalado[y_escalado==-1, 0], X_escalado[y_escalado==-1, 1],_
↳color='#e65100', label='Classe -1')
ax1.set_title('SEM Padronizacao: Colapso Geometrico', fontweight='bold')
ax1.set_xlabel('Idade (18-65)')
ax1.set_ylabel('Renda (10k-150k)')

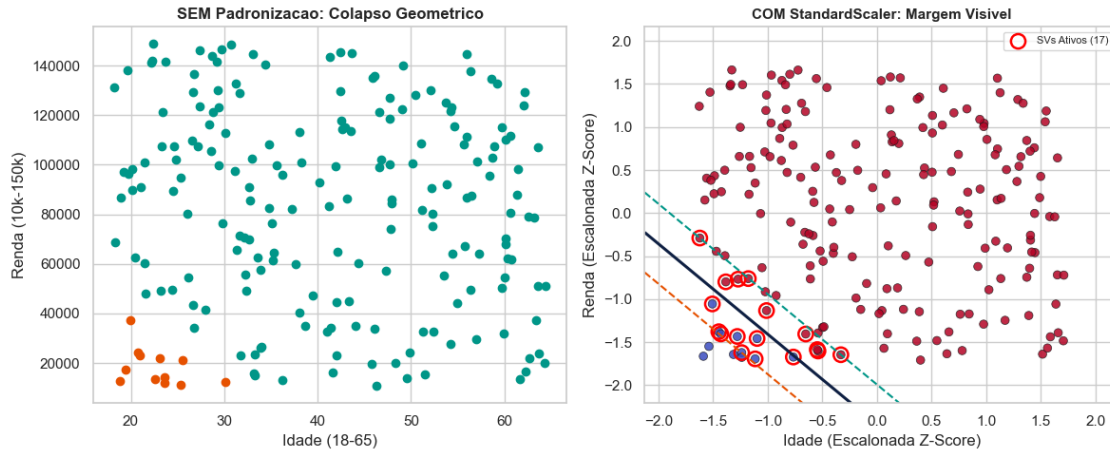
plot_svm_decision_boundary(svm_padronizado, X_escalonado, y_escalado, ax2,_
↳title='COM StandardScaler: Margem Visivel')
ax2.set_xlabel('Idade (Escalonada Z-Score)')
ax2.set_ylabel('Renda (Escalonada Z-Score)')

plt.tight_layout()
plt.show()

```

Linha 1 Crua : Idade 35.6 | Renda 99884.4
 Linha 1 Escalonada : Idade -0.372 | Renda 0.471

Sobrecarga de RAM SEM Padronizacao : 5 Vetores de Suporte (Travamento)
 Eficiencia COM Padronizacao : 17 Vetores de Suporte (Limpo)
 Largura Geometrica (M) Padronizada : 0.6409 unidades



1.2 O Efeito Fisico de C

O parametro C diz ao algoritmo qual o custo de deixar um ponto cair no lado errado da margem.

C baixo (0.01): Multa baixa. A margem alarga. Muitos pontos invadem e viram Vetores de Suporte.

C alto (1000): Multa alta. A margem espreme. So quem encosta na beirada vira Vetor de Suporte.

```
[59]: X_blobs, y_blobs = make_blobs(n_samples=200, centers=2, cluster_std=2.5,
    random_state=42)
y_blobs = np.where(y_blobs == 0, -1, 1)
X_blobs = StandardScaler().fit_transform(X_blobs)

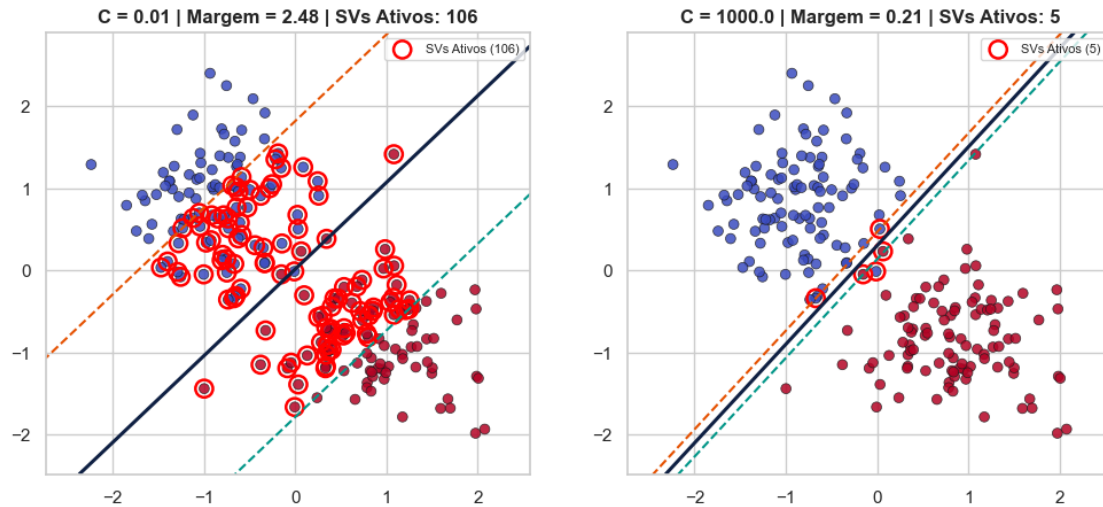
valores_C = [0.01, 1000.0]
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

for ax, C_val in zip(axes, valores_C):
    clf = SVC(kernel='linear', C=C_val)
    clf.fit(X_blobs, y_blobs)

    w_atual = clf.coef_[0]
    espessura = 2 / np.linalg.norm(w_atual)

    plot_svm_decision_boundary(clf, X_blobs, y_blobs, ax, title='')
    ax.set_title(f'C = {C_val} | Margem = {espessura:.2f} | SVs Ativos:
    {len(clf.support_)}', fontweight='bold')

plt.show()
```



```
[60]: X_circ, y_circ = make_circles(n_samples=300, noise=0.15, factor=0.4,
    ↪random_state=42)
y_circ = np.where(y_circ == 0, -1, 1)
X_circ_sc = StandardScaler().fit_transform(X_circ)

linear_svm = SVC(kernel='linear')
linear_svm.fit(X_circ_sc, y_circ)

param_grid = {'C': [0.1, 1, 10], 'gamma': [0.1, 1, 10]}
grid_rbf = GridSearchCV(SVC(kernel='rbf'), param_grid, cv=5)
grid_rbf.fit(X_circ_sc, y_circ)
best_rbf = grid_rbf.best_estimator_

print(f'Busca Completa. Vencedores: {grid_rbf.best_params_}\n')

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
xx, yy = np.meshgrid(np.linspace(-3, 3, 100), np.linspace(-3, 3, 100))

Z_lin = linear_svm.decision_function(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.
    ↪shape)
cont1 = ax1.contourf(xx, yy, Z_lin, cmap='coolwarm', alpha=0.3, levels=50)
fig.colorbar(cont1, ax=ax1, fraction=0.046, pad=0.04)

ax1.scatter(X_circ_sc[:, 0], X_circ_sc[:, 1], c=y_circ, cmap='coolwarm',
    ↪edgecolors='k')
ax1.set_title('Kernel Linear: Falha Geometrica')
```

```

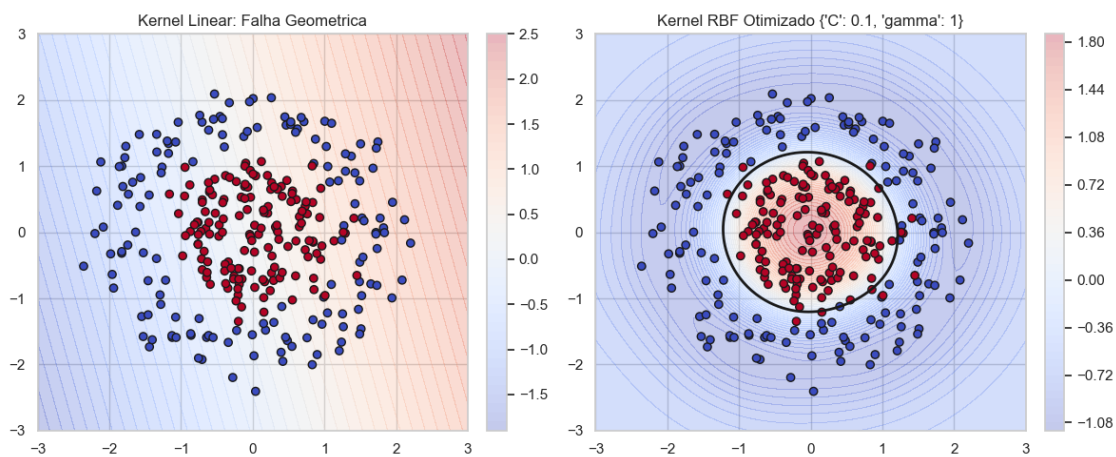
Z_rbf = best_rbf.decision_function(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.
    ↪shape)
cont2 = ax2.contourf(xx, yy, Z_rbf, cmap='coolwarm', alpha=0.3, levels=50)
fig.colorbar(cont2, ax=ax2, fraction=0.046, pad=0.04)

ax2.contour(xx, yy, Z_rbf, colors='k', levels=[0], linewidths=2)
ax2.scatter(X_circ_sc[:, 0], X_circ_sc[:, 1], c=y_circ, cmap='coolwarm',
    ↪edgecolors='k')
ax2.set_title(f'Kernel RBF Otimizado {grid_rbf.best_params_}')

plt.tight_layout()
plt.show()

```

Busca Completa. Vencedores: {'C': 0.1, 'gamma': 1}



1.3 Regressao Continua (epsilon-SVR)

A logica matematica e espelhada: na classificacao, mantemos a margem vazia. Na regressao, desenhamos o **Tubo Insensivel** (com espessura epsilon) engolindo a linha de predicacao. O que cai no tubo tem Custo Zero. O que cai fora do tubo vira um Vetor de Suporte penalizado.

```

[61]: np.random.seed(42)
X_reg = np.sort(np.random.uniform(-3, 3, 100)).reshape(-1, 1)
y_reg = np.sin(X_reg).ravel() + np.random.normal(0, 0.15, 100)

svr = SVR(kernel='rbf', C=10.0, epsilon=0.3)

svr.fit(X_reg, y_reg)

X_plot = np.linspace(-3.2, 3.2, 200).reshape(-1, 1)

```

```

y_pred = svr.predict(X_plot)

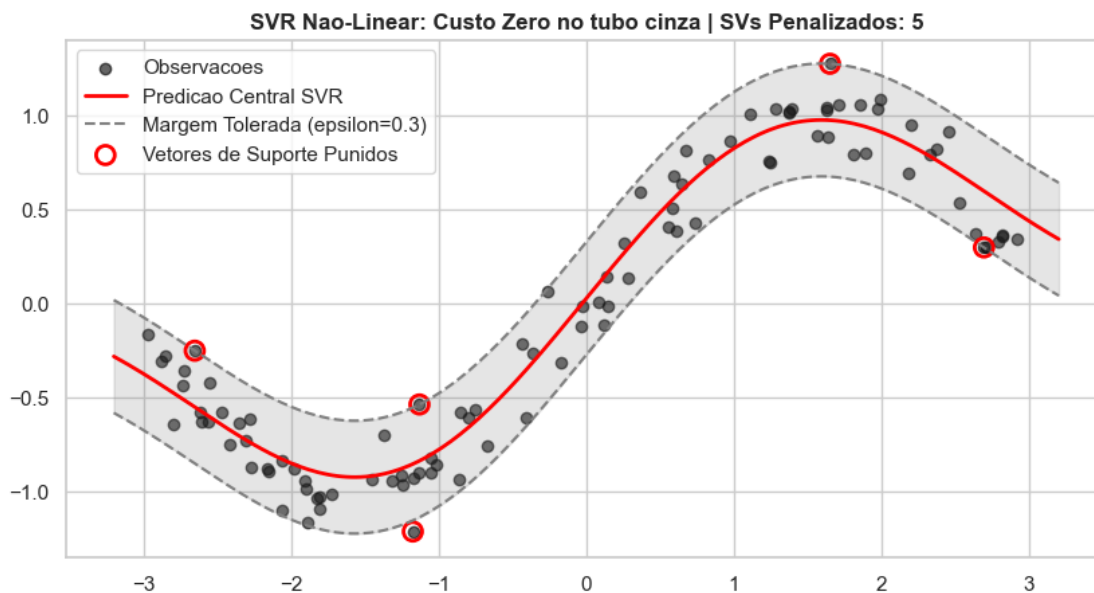
plt.figure(figsize=(10, 5))
plt.scatter(X_reg, y_reg, color='black', label='Observacoes', alpha=0.6)
plt.plot(X_plot, y_pred, color='red', lw=2, label='Predicao Central SVR')

plt.plot(X_plot, y_pred + svr.epsilon, color='gray', linestyle='--')
plt.plot(X_plot, y_pred - svr.epsilon, color='gray', linestyle='--',
        label=f'Margem Tolerada (epsilon={svr.epsilon})')
plt.fill_between(X_plot.ravel(), y_pred - svr.epsilon, y_pred + svr.epsilon,
        color='gray', alpha=0.2)

sv_idx = svr.support_
plt.scatter(X_reg[sv_idx], y_reg[sv_idx], s=100, facecolors='none',
        edgecolors='red', lw=2, label='Vetores de Suporte Punidos')

plt.title(f'SVR Nao-Linear: Custo Zero no tubo cinza | SVs Penalizados: {len(sv_idx)}',
        fontweight='bold')
plt.legend()
plt.show()

```



```

[62]: from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt

```

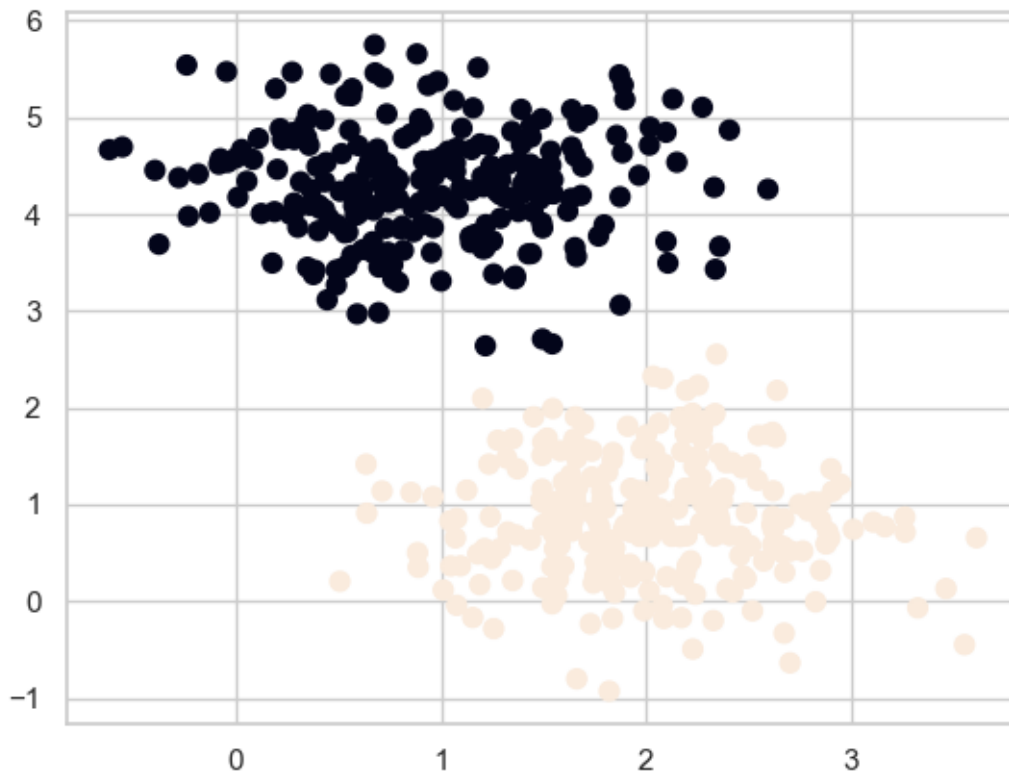
```

[63]: X, Y = make_blobs(n_samples=500, centers=2, cluster_std=0.6
                        ,random_state=0)

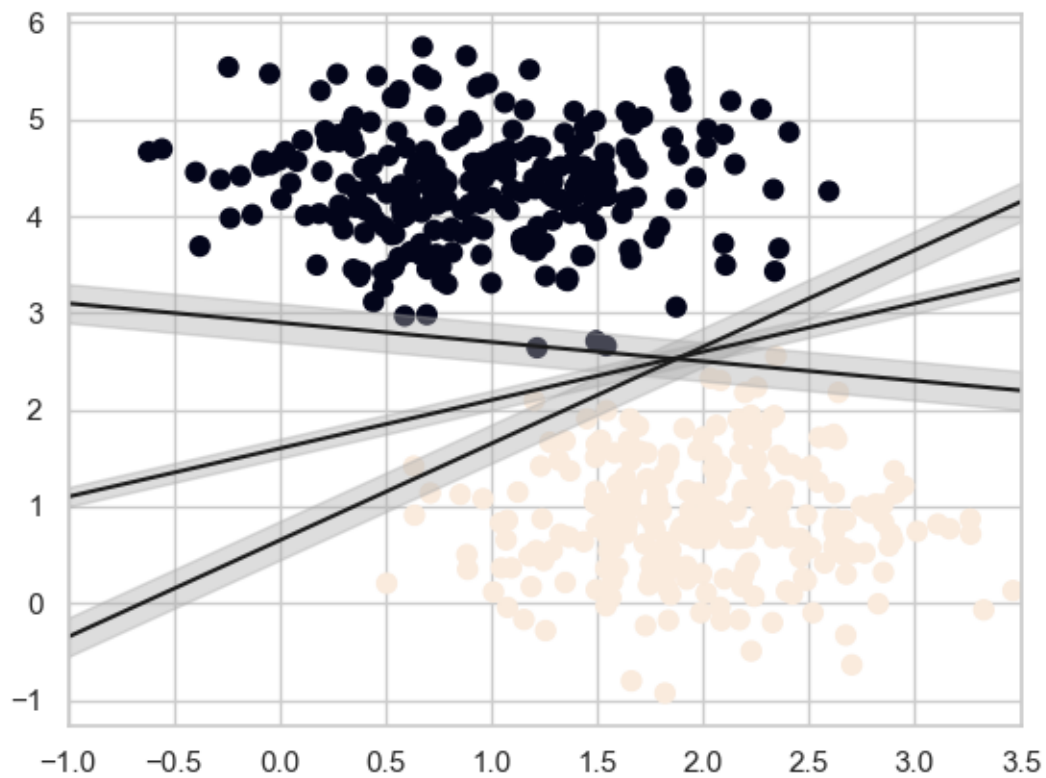
```

```
[64]: plt.scatter(X[:,0], X[:,1], c=Y, s=50)
```

```
[64]: <matplotlib.collections.PathCollection at 0x7ff04997d110>
```

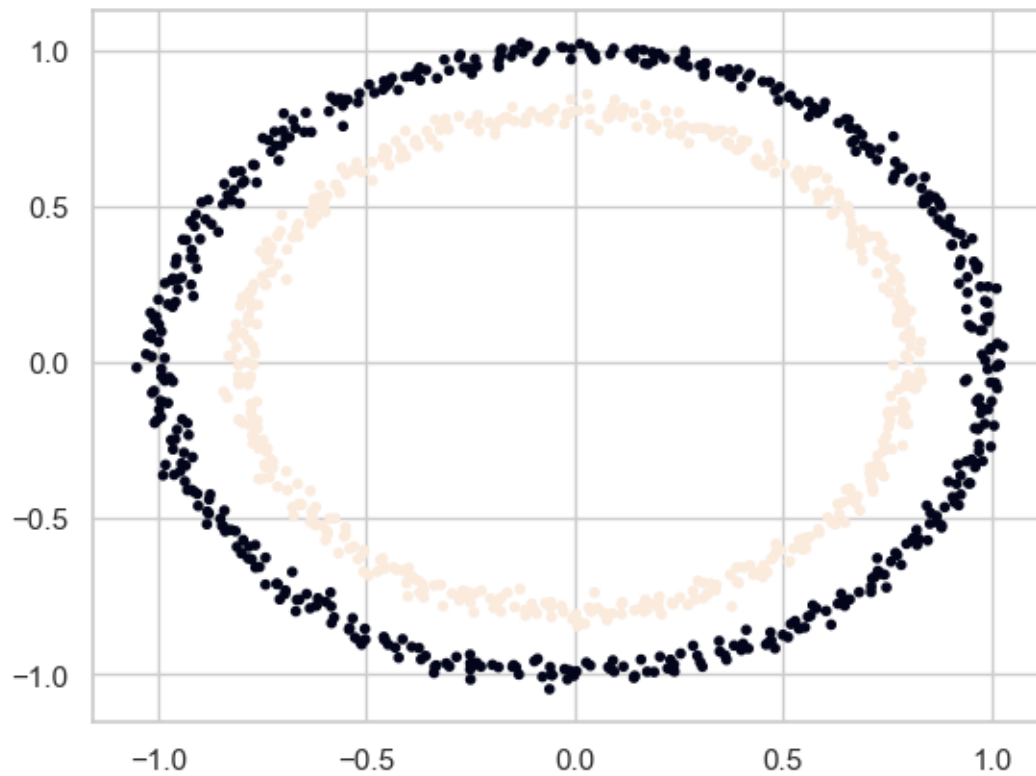


```
[14]: import numpy as np
xfit = np.linspace(-1, 3.5)
plt.scatter(X[:,0], X[:,1], c=Y, s=50)
for m, b, d in [(1, 0.65, 0.2), (0.5, 1.6, 0.1), (-0.2, 2.9, 0.2)]:
    yfit = m*xfit+b
    plt.plot(xfit, yfit, '-k')
    plt.fill_between(xfit, yfit-d, yfit+d, edgecolor='none',
                     color='#AAAAAA', alpha=0.4)
plt.xlim(-1, 3.5)
plt.show()
```



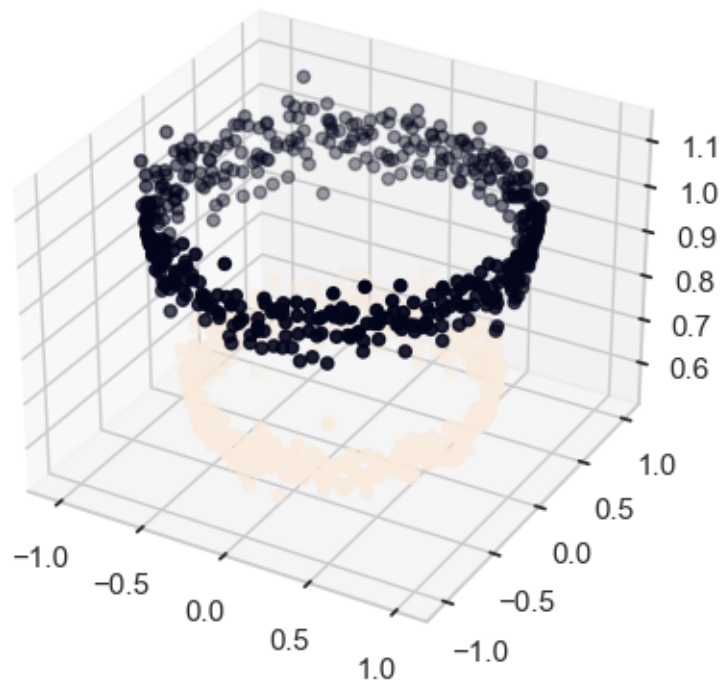
```
[15]: from mpl_toolkits.mplot3d import Axes3D
      from sklearn.datasets import make_circles

      X, Y = make_circles(n_samples=1000, noise=0.02)
      plt.scatter(X[:,0], X[:,1], c=Y, marker='.')
      plt.show()
```



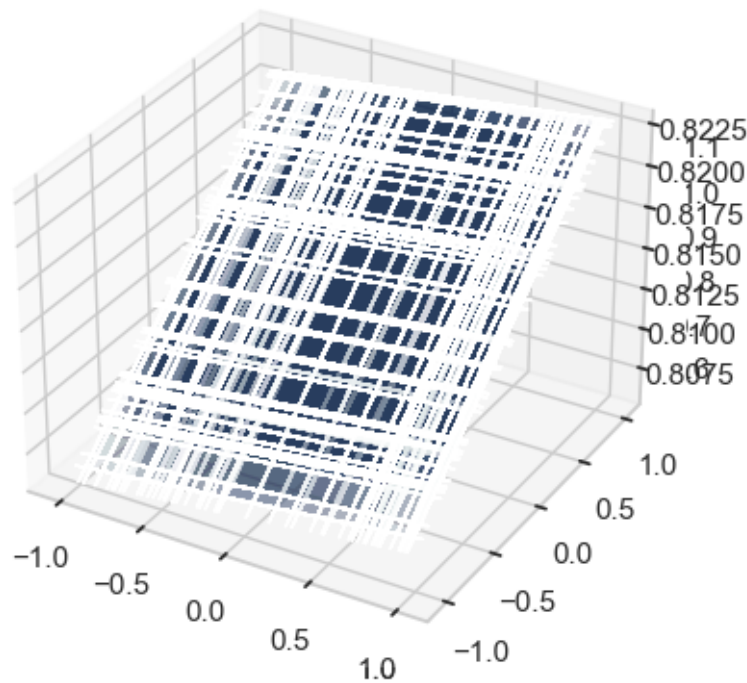
```
[16]: X1=X[:,0].reshape((-1,1))
      X2=X[:,1].reshape((-1,1))
      X3=(X1**2+X2**2)
      X=np.hstack((X,X3))

      fig=plt.figure()
      axes=fig.add_subplot(111, projection='3d')
      axes.scatter(X1, X2, X1**2+X2**2, c=Y, depthshade=True)
      plt.show()
```



```
[19]: from sklearn import svm
svc = svm.SVC(kernel='linear')
svc.fit(X, Y)
w=svc.coef_
b=svc.intercept_
x1=X[:,0].reshape((-1,1))
x2=X[:,1].reshape((-1,1))
x1, x2 = np.meshgrid(x1,x2)
x3 = -(w[0][0]*x1+w[0][1]*x2+b)/w[0][2]
fig = plt.figure()
axes2=fig.add_subplot(111,projection='3d')
axes2.scatter(X1,X2,X1**2+X2**2, c=Y, depthshade=True)
axes1 = fig.add_subplot(projection='3d')
axes1.plot_surface(x1,x2,x3,alpha=0.01)
```

```
[19]: <mpl_toolkits.mplot3d.art3d.Poly3DCollection at 0x7ff049a4d150>
```



```
[20]: from sklearn.datasets import load_breast_cancer
```

```
[21]: cancer = load_breast_cancer()
```

```
[22]: cancer.keys()
```

```
[22]: dict_keys(['data', 'target', 'frame', 'target_names', 'DESCR', 'feature_names',
               'filename', 'data_module'])
```

```
[23]: print(cancer['DESCR'])
```

```
.. _breast_cancer_dataset:
```

```
Breast cancer wisconsin (diagnostic) dataset
```

```
-----
```

```
**Data Set Characteristics:**
```

```
:Number of Instances: 569
```

```
:Number of Attributes: 30 numeric, predictive attributes and the class
```

```
:Attribute Information:
```

```
  - radius (mean of distances from center to points on the perimeter)
```

- texture (standard deviation of gray-scale values)
- perimeter
- area
- smoothness (local variation in radius lengths)
- compactness ($\text{perimeter}^2 / \text{area} - 1.0$)
- concavity (severity of concave portions of the contour)
- concave points (number of concave portions of the contour)
- symmetry
- fractal dimension ("coastline approximation" - 1)

The mean, standard error, and "worst" or largest (mean of the three worst/largest values) of these features were computed for each image, resulting in 30 features. For instance, field 0 is Mean Radius, field 10 is Radius SE, field 20 is Worst Radius.

- class:
 - WDBC-Malignant
 - WDBC-Benign

:Summary Statistics:

	Min	Max
radius (mean):	6.981	28.11
texture (mean):	9.71	39.28
perimeter (mean):	43.79	188.5
area (mean):	143.5	2501.0
smoothness (mean):	0.053	0.163
compactness (mean):	0.019	0.345
concavity (mean):	0.0	0.427
concave points (mean):	0.0	0.201
symmetry (mean):	0.106	0.304
fractal dimension (mean):	0.05	0.097
radius (standard error):	0.112	2.873
texture (standard error):	0.36	4.885
perimeter (standard error):	0.757	21.98
area (standard error):	6.802	542.2
smoothness (standard error):	0.002	0.031
compactness (standard error):	0.002	0.135
concavity (standard error):	0.0	0.396
concave points (standard error):	0.0	0.053
symmetry (standard error):	0.008	0.079
fractal dimension (standard error):	0.001	0.03
radius (worst):	7.93	36.04
texture (worst):	12.02	49.54
perimeter (worst):	50.41	251.2
area (worst):	185.2	4254.0

smoothness (worst):	0.071	0.223
compactness (worst):	0.027	1.058
concavity (worst):	0.0	1.252
concave points (worst):	0.0	0.291
symmetry (worst):	0.156	0.664
fractal dimension (worst):	0.055	0.208
=====	=====	=====

:Missing Attribute Values: None

:Class Distribution: 212 - Malignant, 357 - Benign

:Creator: Dr. William H. Wolberg, W. Nick Street, Olvi L. Mangasarian

:Donor: Nick Street

:Date: November, 1995

This is a copy of UCI ML Breast Cancer Wisconsin (Diagnostic) datasets.
<https://goo.gl/U2Uwz2>

Features are computed from a digitized image of a fine needle aspirate (FNA) of a breast mass. They describe characteristics of the cell nuclei present in the image.

Separating plane described above was obtained using Multisurface Method-Tree (MSM-T) [K. P. Bennett, "Decision Tree Construction Via Linear Programming." Proceedings of the 4th Midwest Artificial Intelligence and Cognitive Science Society, pp. 97-101, 1992], a classification method which uses linear programming to construct a decision tree. Relevant features were selected using an exhaustive search in the space of 1-4 features and 1-3 separating planes.

The actual linear program used to obtain the separating plane in the 3-dimensional space is that described in: [K. P. Bennett and O. L. Mangasarian: "Robust Linear Programming Discrimination of Two Linearly Inseparable Sets", Optimization Methods and Software 1, 1992, 23-34].

This database is also available through the UW CS ftp server:

```
ftp ftp.cs.wisc.edu
cd math-prog/cpo-dataset/machine-learn/WDBC/
```

.. topic:: References

- W.N. Street, W.H. Wolberg and O.L. Mangasarian. Nuclear feature extraction

for breast tumor diagnosis. IS&T/SPIE 1993 International Symposium on Electronic Imaging: Science and Technology, volume 1905, pages 861-870, San Jose, CA, 1993.

- O.L. Mangasarian, W.N. Street and W.H. Wolberg. Breast cancer diagnosis and prognosis via linear programming. Operations Research, 43(4), pages 570-577, July-August 1995.
- W.H. Wolberg, W.N. Street, and O.L. Mangasarian. Machine learning techniques to diagnose breast cancer from fine-needle aspirates. Cancer Letters 77 (1994) 163-171.

```
[24]: import pandas as pd
```

```
[25]: df = pd.DataFrame(cancer['data'], columns=cancer['feature_names'])
```

```
[27]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 30 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   mean radius                           569 non-null    float64
1   mean texture                           569 non-null    float64
2   mean perimeter                         569 non-null    float64
3   mean area                             569 non-null    float64
4   mean smoothness                       569 non-null    float64
5   mean compactness                      569 non-null    float64
6   mean concavity                         569 non-null    float64
7   mean concave points                   569 non-null    float64
8   mean symmetry                         569 non-null    float64
9   mean fractal dimension                 569 non-null    float64
10  radius error                           569 non-null    float64
11  texture error                           569 non-null    float64
12  perimeter error                         569 non-null    float64
13  area error                             569 non-null    float64
14  smoothness error                       569 non-null    float64
15  compactness error                      569 non-null    float64
16  concavity error                        569 non-null    float64
17  concave points error                   569 non-null    float64
18  symmetry error                         569 non-null    float64
19  fractal dimension error                 569 non-null    float64
20  worst radius                           569 non-null    float64
21  worst texture                           569 non-null    float64
22  worst perimeter                         569 non-null    float64
23  worst area                             569 non-null    float64
```

```

24  worst smoothness      569 non-null    float64
25  worst compactness     569 non-null    float64
26  worst concavity       569 non-null    float64
27  worst concave points  569 non-null    float64
28  worst symmetry        569 non-null    float64
29  worst fractal dimension 569 non-null    float64
dtypes: float64(30)
memory usage: 133.5 KB

```

```
[28]: cancer['target']
```

```

[28]: array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
0, 0, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 0, 0,
1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 0, 0,
1, 1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1,
1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 0,
0, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1,
1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 1,
1, 0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 0, 0,
0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0,
1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 1,
1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 1, 0, 0,
0, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0,
0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0,
1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 1, 0, 0, 1, 1,
1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 0,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 1, 1,
1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0,
1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1,
1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 1, 0, 1, 1,
1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1])

```

```
[29]: df_target=pd.DataFrame(cancer['target'], columns=['Cancer'])
```

```
[30]: df_target.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 1 columns):
#   Column  Non-Null Count  Dtype
---  -
0    Cancer  569 non-null      int64

```

```
dtypes: int64(1)
memory usage: 4.6 KB
```

```
[31]: df.describe().T
```

```
[31]:
```

	count	mean	std	min \
mean radius	569.0	14.127292	3.524049	6.981000
mean texture	569.0	19.289649	4.301036	9.710000
mean perimeter	569.0	91.969033	24.298981	43.790000
mean area	569.0	654.889104	351.914129	143.500000
mean smoothness	569.0	0.096360	0.014064	0.052630
mean compactness	569.0	0.104341	0.052813	0.019380
mean concavity	569.0	0.088799	0.079720	0.000000
mean concave points	569.0	0.048919	0.038803	0.000000
mean symmetry	569.0	0.181162	0.027414	0.106000
mean fractal dimension	569.0	0.062798	0.007060	0.049960
radius error	569.0	0.405172	0.277313	0.111500
texture error	569.0	1.216853	0.551648	0.360200
perimeter error	569.0	2.866059	2.021855	0.757000
area error	569.0	40.337079	45.491006	6.802000
smoothness error	569.0	0.007041	0.003003	0.001713
compactness error	569.0	0.025478	0.017908	0.002252
concavity error	569.0	0.031894	0.030186	0.000000
concave points error	569.0	0.011796	0.006170	0.000000
symmetry error	569.0	0.020542	0.008266	0.007882
fractal dimension error	569.0	0.003795	0.002646	0.000895
worst radius	569.0	16.269190	4.833242	7.930000
worst texture	569.0	25.677223	6.146258	12.020000
worst perimeter	569.0	107.261213	33.602542	50.410000
worst area	569.0	880.583128	569.356993	185.200000
worst smoothness	569.0	0.132369	0.022832	0.071170
worst compactness	569.0	0.254265	0.157336	0.027290
worst concavity	569.0	0.272188	0.208624	0.000000
worst concave points	569.0	0.114606	0.065732	0.000000
worst symmetry	569.0	0.290076	0.061867	0.156500
worst fractal dimension	569.0	0.083946	0.018061	0.055040

	25%	50%	75%	max
mean radius	11.700000	13.370000	15.780000	28.11000
mean texture	16.170000	18.840000	21.800000	39.28000
mean perimeter	75.170000	86.240000	104.100000	188.50000
mean area	420.300000	551.100000	782.700000	2501.00000
mean smoothness	0.086370	0.095870	0.105300	0.16340
mean compactness	0.064920	0.092630	0.130400	0.34540
mean concavity	0.029560	0.061540	0.130700	0.42680
mean concave points	0.020310	0.033500	0.074000	0.20120
mean symmetry	0.161900	0.179200	0.195700	0.30400

mean fractal dimension	0.057700	0.061540	0.066120	0.09744
radius error	0.232400	0.324200	0.478900	2.87300
texture error	0.833900	1.108000	1.474000	4.88500
perimeter error	1.606000	2.287000	3.357000	21.98000
area error	17.850000	24.530000	45.190000	542.20000
smoothness error	0.005169	0.006380	0.008146	0.03113
compactness error	0.013080	0.020450	0.032450	0.13540
concavity error	0.015090	0.025890	0.042050	0.39600
concave points error	0.007638	0.010930	0.014710	0.05279
symmetry error	0.015160	0.018730	0.023480	0.07895
fractal dimension error	0.002248	0.003187	0.004558	0.02984
worst radius	13.010000	14.970000	18.790000	36.04000
worst texture	21.080000	25.410000	29.720000	49.54000
worst perimeter	84.110000	97.660000	125.400000	251.20000
worst area	515.300000	686.500000	1084.000000	4254.00000
worst smoothness	0.116600	0.131300	0.146000	0.22260
worst compactness	0.147200	0.211900	0.339100	1.05800
worst concavity	0.114500	0.226700	0.382900	1.25200
worst concave points	0.064930	0.099930	0.161400	0.29100
worst symmetry	0.250400	0.282200	0.317900	0.66380
worst fractal dimension	0.071460	0.080040	0.092080	0.20750

```
[32]: from sklearn.preprocessing import StandardScaler
```

```
[33]: std = StandardScaler()
```

```
[34]: std.fit(df)
```

```
[34]: StandardScaler()
```

```
[35]: X_std = std.transform(df)
```

```
[36]: from sklearn.model_selection import train_test_split
```

```
[37]: X_treino, X_test, Y_treino, Y_test = train_test_split(X_std, np.
    ↪ ravel(df_target),
    test_size=0.3,
    ↪ random_state=101)
```

```
[38]: model = SVC()
model.fit(X_treino, Y_treino)
```

```
[38]: SVC()
```

```
[39]: Y_predito = model.predict(X_test)
```

```
[40]: print(classification_report(Y_test, Y_predito))
```

	precision	recall	f1-score	support
0	0.98	0.95	0.97	66
1	0.97	0.99	0.98	105
accuracy			0.98	171
macro avg	0.98	0.97	0.98	171
weighted avg	0.98	0.98	0.98	171

```
[41]: paramatro_grid={'C':[0.1,1,10,100,1000], 'gamma':[1, 0.1, 0.01, 0.001, 0.0001],
                    'kernel':['rbf']}
from sklearn.model_selection import GridSearchCV
```

```
[42]: grid = GridSearchCV(SVC(), paramatro_grid, refit=True, verbose=3)
```

```
[43]: grid.fit(X_treino, Y_treino)
```

Fitting 5 folds for each of 25 candidates, totalling 125 fits

```
[CV 1/5] END ...C=0.1, gamma=1, kernel=rbf;; score=0.637 total time= 0.0s
[CV 2/5] END ...C=0.1, gamma=1, kernel=rbf;; score=0.637 total time= 0.0s
[CV 3/5] END ...C=0.1, gamma=1, kernel=rbf;; score=0.625 total time= 0.0s
[CV 4/5] END ...C=0.1, gamma=1, kernel=rbf;; score=0.633 total time= 0.0s
[CV 5/5] END ...C=0.1, gamma=1, kernel=rbf;; score=0.633 total time= 0.0s
[CV 1/5] END ...C=0.1, gamma=0.1, kernel=rbf;; score=0.925 total time= 0.0s
[CV 2/5] END ...C=0.1, gamma=0.1, kernel=rbf;; score=0.950 total time= 0.0s
[CV 3/5] END ...C=0.1, gamma=0.1, kernel=rbf;; score=0.900 total time= 0.0s
[CV 4/5] END ...C=0.1, gamma=0.1, kernel=rbf;; score=0.962 total time= 0.0s
[CV 5/5] END ...C=0.1, gamma=0.1, kernel=rbf;; score=0.949 total time= 0.0s
[CV 1/5] END ...C=0.1, gamma=0.01, kernel=rbf;; score=0.912 total time= 0.0s
[CV 2/5] END ...C=0.1, gamma=0.01, kernel=rbf;; score=0.963 total time= 0.0s
[CV 3/5] END ...C=0.1, gamma=0.01, kernel=rbf;; score=0.975 total time= 0.0s
[CV 4/5] END ...C=0.1, gamma=0.01, kernel=rbf;; score=0.987 total time= 0.0s
[CV 5/5] END ...C=0.1, gamma=0.01, kernel=rbf;; score=0.962 total time= 0.0s
[CV 1/5] END ...C=0.1, gamma=0.001, kernel=rbf;; score=0.688 total time= 0.0s
[CV 2/5] END ...C=0.1, gamma=0.001, kernel=rbf;; score=0.688 total time= 0.0s
[CV 3/5] END ...C=0.1, gamma=0.001, kernel=rbf;; score=0.688 total time= 0.0s
[CV 4/5] END ...C=0.1, gamma=0.001, kernel=rbf;; score=0.684 total time= 0.0s
[CV 5/5] END ...C=0.1, gamma=0.001, kernel=rbf;; score=0.709 total time= 0.0s
[CV 1/5] END ...C=0.1, gamma=0.0001, kernel=rbf;; score=0.637 total time= 0.0s
[CV 2/5] END ...C=0.1, gamma=0.0001, kernel=rbf;; score=0.637 total time= 0.0s
[CV 3/5] END ...C=0.1, gamma=0.0001, kernel=rbf;; score=0.625 total time= 0.0s
[CV 4/5] END ...C=0.1, gamma=0.0001, kernel=rbf;; score=0.633 total time= 0.0s
[CV 5/5] END ...C=0.1, gamma=0.0001, kernel=rbf;; score=0.633 total time= 0.0s
[CV 1/5] END ...C=1, gamma=1, kernel=rbf;; score=0.637 total time= 0.0s
[CV 2/5] END ...C=1, gamma=1, kernel=rbf;; score=0.637 total time= 0.0s
[CV 3/5] END ...C=1, gamma=1, kernel=rbf;; score=0.625 total time= 0.0s
[CV 4/5] END ...C=1, gamma=1, kernel=rbf;; score=0.633 total time= 0.0s
```

[illegible]

[illegible]

```
[43]: GridSearchCV(estimator=SVC(),
                  param_grid={'C': [0.1, 1, 10, 100, 1000],
                              'gamma': [1, 0.1, 0.01, 0.001, 0.0001],
                              'kernel': ['rbf']},
                  verbose=3)
```

```
[44]: grid.best_params_
```

```
[44]: {'C': 10, 'gamma': 0.01, 'kernel': 'rbf'}
```

```
[45]: grid_predictor=grid.predict(X_test)
```

```
[47]: print(classification_report(Y_test, grid_predictor))
```

	precision	recall	f1-score	support
0	1.00	0.97	0.98	66
1	0.98	1.00	0.99	105
accuracy			0.99	171
macro avg	0.99	0.98	0.99	171
weighted avg	0.99	0.99	0.99	171

```
[48]: from sklearn.metrics import classification_report, confusion_matrix
```

```
[49]: print(confusion_matrix(Y_test, Y_predicto))
```

```
[[ 63   3]
 [   1 104]]
```

```
[50]: print(confusion_matrix(Y_test, grid_predictor))
```

```
[[ 64   2]
 [   0 105]]
```

```
[ ]:
```